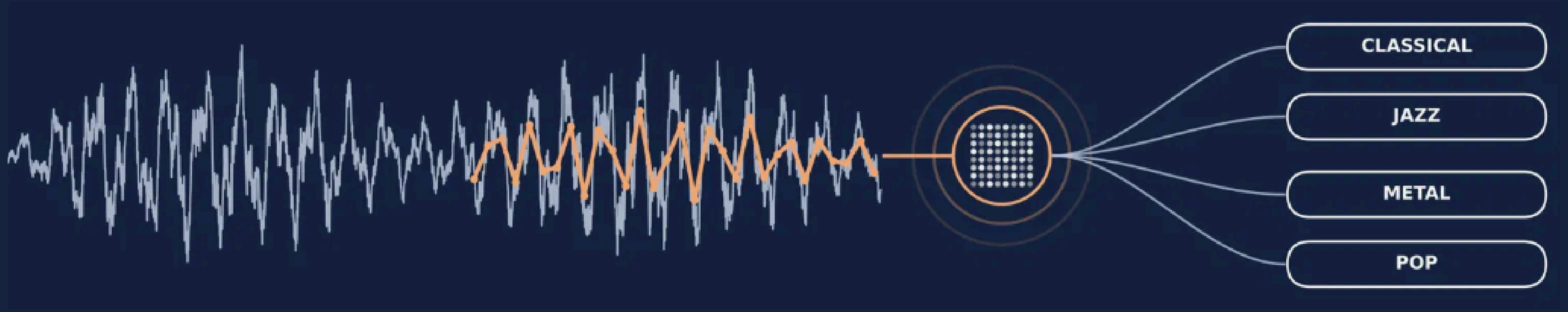
UE24CS352A MACHINE LEARNING · MINI-PROJECT

Latent Feature Extraction for Musical Genres from Raw Audio

Paper: Sawhney, Vasavada & Wang (Stanford, 2018) · Dataset: GTZAN



Harsha K - PES1UG24CS666
Sunil - PES1UG24CS634

github.com/Harsha-Kamaraj/ML-Mini-Project

THE PROBLEM

Can a model guess genre from raw sound?

INPUT

A short piece of a song, as raw sound numbers. No extra audio features.

OUTPUT

The genre (classical, jazz, metal or pop) and a 64-number summary of the clip

PAPER'S RESULT

65.3%

of test clips guessed right by their best model

Our task ① Rebuild the paper exactly ② Make the model better

DATASET · GTZAN

400 songs turned into 8,000 small examples

1 song

30 seconds of sound, read as a long list of numbers using LibROSA

→

20 pieces

each about 0.9 seconds long

→

500 numbers

average of every 40 numbers

→

8,000

examples, 2,000 per genre

4 genres × 100 songs

Classical, jazz, metal, pop. 6,000 to train, 1,000 to tune, 1,000 to test.

Checked the audio

The original website is down, so we used a copy and checked every file.

PART 1 · REBUILDING THE PAPER

The paper's three models

Two-layer NN

A simple classifier: 500 inputs → 128 → 4 genres. Our starting point.

Vanilla autoencoder

Squeezes the clip down to **64** numbers, then rebuilds it. Never sees the genre labels.

Deep Softmax AE

The autoencoder plus a small genre classifier on the 64 numbers. The paper's main model.

Loss: 90% "rebuild the sound well" + 10% "guess the genre right"

Trained with Adam · learning rate 0.0001 · 512 clips per batch · 10% dropout

Rule: follow the paper first; use the authors' code only where the paper says nothing

PART 1 · DID WE GET THE SAME RESULTS?

Simple model matches; main model is about 4% lower

Accuracy	Paper	Ours (run 1)	Ours, average of 5 runs
Two-layer NN · test	36.4%	38.9%	39.0% (37.2–41.0)
Deep Softmax AE · train	94.9%	96.2%	94.8% (93.0–96.2)
Deep Softmax AE · test	65.3%	57.7%	61.0% (57.7–63.9)

Same pattern as the paper

Classical is easiest. Metal gets mixed up with pop, and classical with jazz.

Every number is our own

We ran everything ourselves. Paper numbers are shown only to compare.

PART 1 · WHY OUR NUMBERS ARE LOWER

Three reasons we found

±6 pts

Which clips get tested

Changing only the random split moves test accuracy from 57.7% to 63.9%.

85.4%

Code ≠ paper

The authors' own code reaches only 85.4% train, not the 94.9% they reported.

0%

Broken decoder

Its last layer can't output negative values, so it gets stuck at 0. The rebuild part does nothing.

Could not test: hidden tuning, picking their best run, TensorFlow vs PyTorch differences.

PART 2 · TESTING FAIRLY

Each model is tested two ways, five times

Clip split · the paper's way

Clips are mixed randomly, so other clips of a test song are seen in training.

Use: compare with the paper

Song split · our way

Whole songs kept together: 320 to train, 40 to tune, 40 to test.

Use: fair score on songs never heard

5 runs → report the average

Tuning set chooses every setting

Test set used only at the end

OUR MAIN MODEL · STEP 3

Convolutional Deep Softmax Autoencoder

Raw sound
5,000 numbers per clip (keeps sounds up to 2,750 Hz)

→

4 conv blocks
small filters that slide along the sound

→

64-number summary
the main patterns found in the clip

→

Genre · same 64→32→16→4 classifier

Rebuilt sound · reverse conv layers

Why convolution works
One small pattern finder slides across the whole clip, so it spots a drum hit or a note wherever it happens.

87.2% / 83.2%
on paper's split / new songs (paper: 65.3%)

EXTRA MODEL · STEP 4

Mel Spectrogram Deep Softmax Autoencoder

Raw sound
the same 20,000-number clip (about 0.9 seconds)

→

Mel spectrogram
a 64 × 40 picture: pitch up, time across

→

64-number summary
from 4 blocks of 2-D conv layers

→

Genre · same 64→32→16→4 classifier

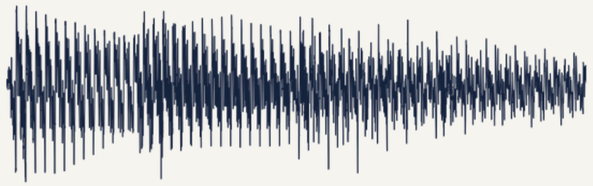
Rebuilt picture · reverse conv layers

Why it works so well

Genres differ in which pitches are loud and how they change. The picture shows this directly and keeps the high sounds.

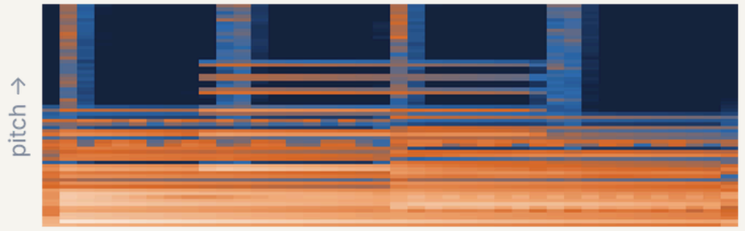
97.7% / 94.5%

on paper's split / new songs (paper: 65.3%)



1. Raw sound · 20,000 numbers

40 time steps × 64 pitch bands →



2. Mel spectrogram · 64 × 40 picture

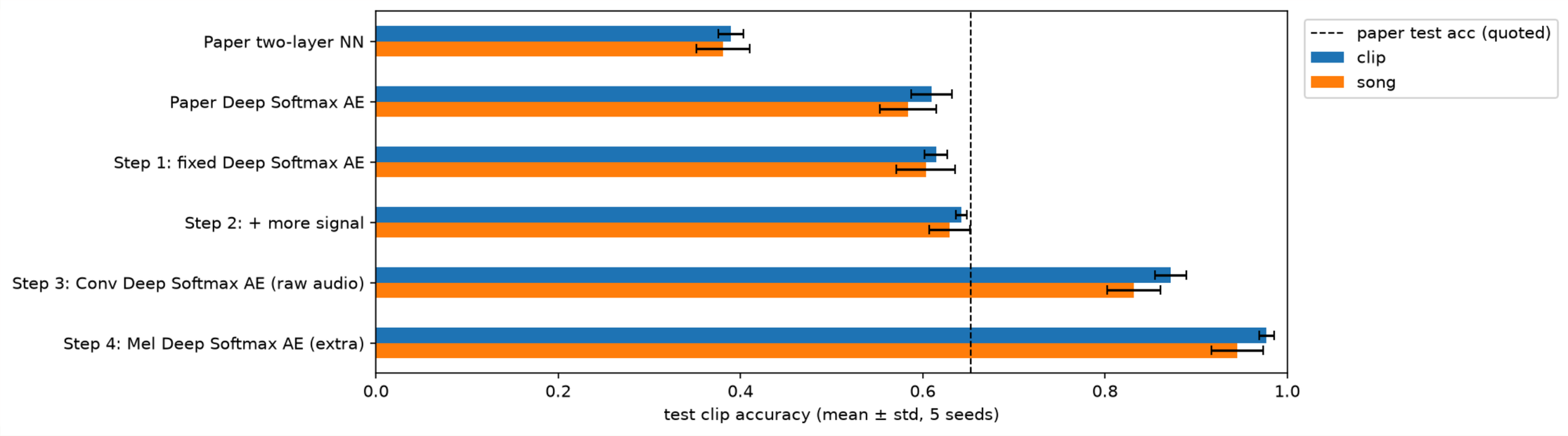
Horizontal stripes
held notes and bass

Vertical stripes
drum hits

Mel scale
more rows for low notes

RESULTS

Our model beats the paper by 22 points



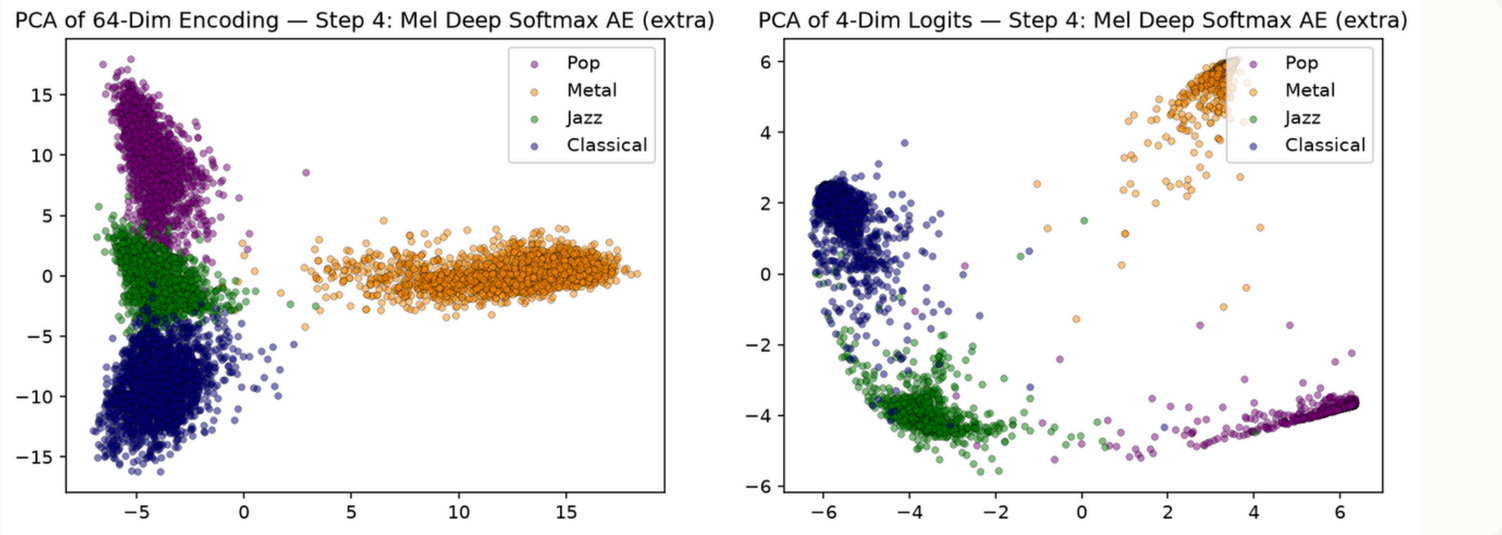
Blue = paper's split

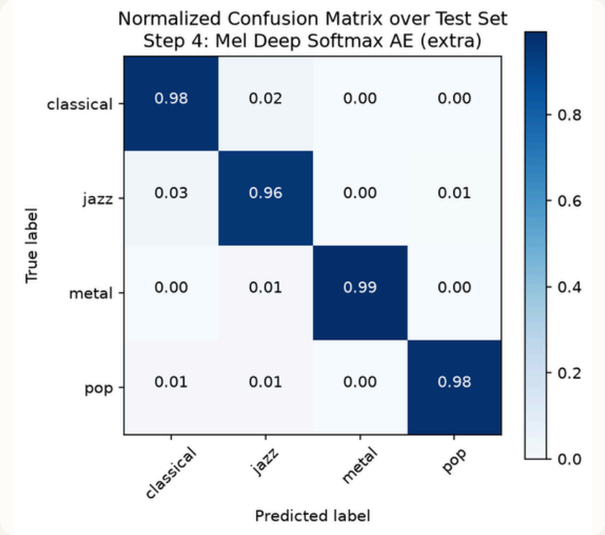
Orange = new songs

Dashed = paper's 65.3%

BEST MODEL · STEP 4, MEL SPECTROGRAM

Each genre forms its own clear group





Test F1 per genre	Classical	Jazz	Metal	Pop
Ours	0.97	0.96	0.99	0.99
Paper	0.78	0.62	0.53	0.64

FINAL COMPARISON

All five models side by side

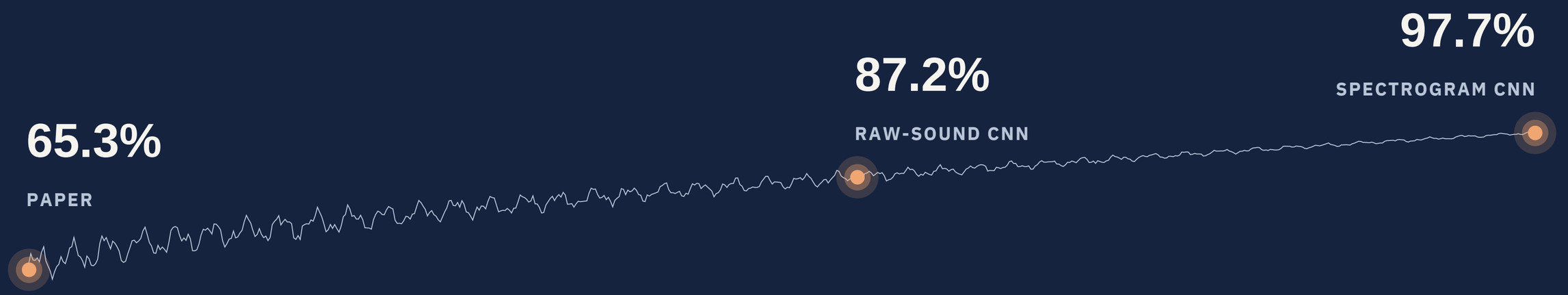
Model	What we changed	Train	Dev	Test
Paper's model (ours)	nothing, just the rebuilt paper model	94.8%	59.9%	61.0%
1 · Fix the model	fixed the broken decoder and training	93.0%	63.2%	61.5%
2 · More sound	used all 30 seconds of every song for training	92.0%	65.4%	64.3%
3 · Conv autoencoder	convolution layers on raw sound	99.2%	88.6%	87.2%
4 · Mel autoencoder (extra)	sound turned into a frequency picture first	99.5%	98.1%	97.7%

Accuracy on the paper's split, average of 5 runs. The paper reported 94.9% train, 64.1% dev, 65.3% test.

UE24CS352A MACHINE LEARNING · MINI-PROJECT

Thank you

From raw sound to genre: 65.3% → 97.7%



Harsha K - PES1UG24CS666
Sunil - PES1UG24CS634

github.com/Harsha-Kamaraj/ML-Mini-Project

Questions?